OFFLINE-FIRST COMMUNICATION PLATFORM FOR OUTDOOR AND EVENT MANAGEMENT

R26-IT-135

Project Proposal Report

Dulsara H G N — IT22544490

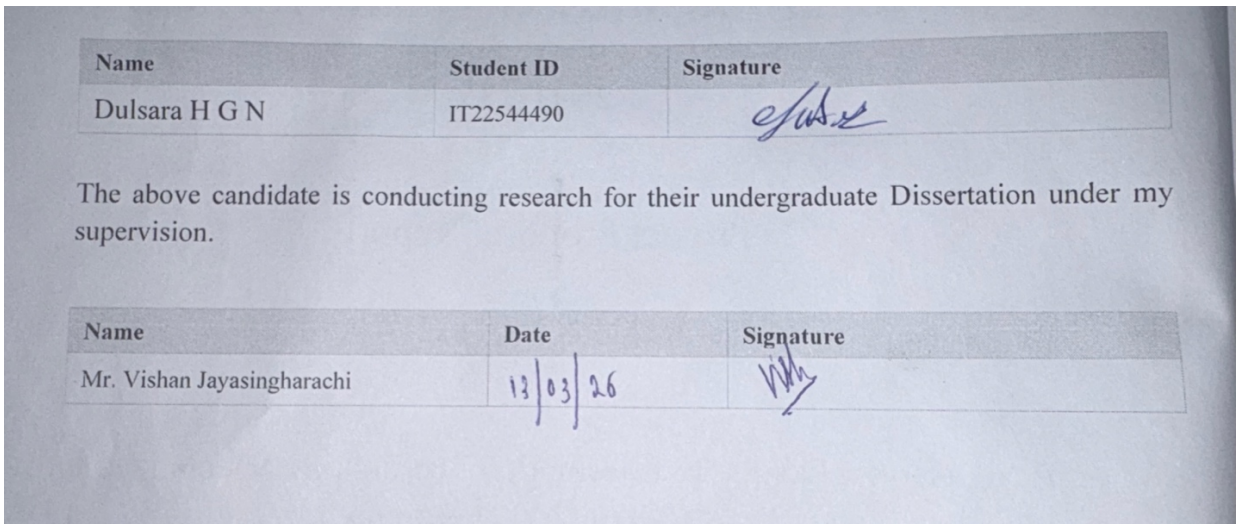
B.Sc. (Hons) Degree in Information Technology Specialized in Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology Sri Lanka

March 2026

DECLARATION

I declare that this is my own work. This proposal does not include any material previously submitted for a degree at another institution, nor any material published or written by another person, except where proper acknowledgment is given.



Name	Student ID	Signature
Dulsara H G N	IT22544490	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Mr. Vishan Jayasingharachi	13/03/26	

ABSTRACT

Millions of people face situations where mobile networks are unavailable — on remote hiking trails, at outdoor events, or in disaster zones. When this happens, standard messaging apps such as WhatsApp and Telegram stop working completely because they need the internet. Existing offline tools like GoTenna and Bridgefy use Bluetooth mesh technology but have major weaknesses: they use fixed routing rules, ignore device battery levels, have no emergency message priority, and one (Bridgefy) was found to have serious security flaws [4].

This research develops a Context-Aware Communication Manager (CACM) — the decision-making core of an Offline-First Communication Platform for Outdoor and Event Management. The identified research gap is clear: no existing offline mesh communication system includes an intelligent layer that chooses the best communication path based on real-time device conditions. The CACM fills this gap using a lightweight rule-based engine that checks five conditions per message — emergency priority, device proximity, relay battery level, queue size, and connectivity status — and routes each message through the most suitable channel: BLE direct messaging (0–30m), multi-hop relay routing (30–100m), or delay-tolerant store-and-forward (100m+).

The system runs on standard Android smartphones using native Kotlin and Bluetooth Low Energy, with no dedicated hardware or internet needed. Evaluation uses real device testing with metrics including routing accuracy, emergency delivery speed, and battery savings. The expected outcome is measurably better routing performance compared to non-adaptive systems, directly supporting disaster communication, outdoor safety, and event coordination. This work advances SDG 3 (Health), SDG 9 (Infrastructure), and SDG 11 (Sustainable Cities).

Keywords: Context-Aware Routing, Offline Communication, BLE Mesh, Delay Tolerant Networks, Emergency Prioritization, Rule-Based Decision Engine

ACKNOWLEDGEMENT

I sincerely thank my supervisor Mr. Vishan Jayasingharachi for his guidance and feedback throughout this research. His knowledge of software systems and mobile networking helped shape the design of the Context-Aware Communication Manager.

I also thank my project team members (IT22148704, IT22168290, IT22894038) for their close collaboration. Understanding their components — BLE Direct, Multi-Hop Routing, and DTN Transport — was essential for designing the CACM that coordinates all three. Finally, I thank the Department of Information Technology at SLIIT for providing the academic resources and support needed for this research.

TABLE OF CONTENT (Auto Generated)

DECLARATION 2
ABSTRACT 2
ACKNOWLEDGEMENT 4
TABLE OF CONTENT (Auto Generated) 5
LIST OF FIGURES 6
LIST OF ABBREVIATIONS 6
LIST OF APPENDICES 7
INTRODUCTION 8
1.1 Background and Context 8
1.2 Literature Review 8
1.3 Research Gap 10
OBJECTIVE 11
2.1 Main Objective 11
2.2 Specific Objectives 11
METHODOLOGY 12
HIGH-LEVEL SYSTEM ARCHITECTURE 17
USER REQUIREMENTS 21
COMMERCIALIZATION PLAN 23
BUDGET AND JUSTIFICATION 24
WORK BREAKDOWN STRUCTURE (WBS) 25
GANTT CHART 26
REFERENCES LIST 27
APPENDICES 28
Appendix A: Decision Engine Step-by-Step Flowchart 28
Appendix B: Hello Packet Structure 28

LIST OF FIGURES

Figure 1: Overall System Architecture — Offline-First Communication Platform

Figure 2: Context-Aware Communication Manager — Internal Architecture

Figure 3: Work Breakdown Structure (WBS)

Figure 4: Project Gantt Chart (March 2026 – July 2026)

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ACK	Acknowledgement
API	Application Programming Interface
BLE	Bluetooth Low Energy
CACM	Context-Aware Communication Manager
DTN	Delay Tolerant Network
FR	Functional Requirement
MANET	Mobile Ad-hoc Network
NFR	Non-Functional Requirement
RSSI	Received Signal Strength Indicator
SCF	Store-Carry-Forward
SDG	Sustainable Development Goal
TTL	Time-To-Live
UUID	Universally Unique Identifier
WBS	Work Breakdown Structure

LIST OF APPENDICES

Appendix A: Decision Engine Step-by-Step Flowchart

Appendix B: Hello Packet Structure

Appendix C: Risk Register

Appendix D: Inter-Component API Specification

INTRODUCTION

1.1 Background and Context

Communication failures in outdoor and emergency settings put lives at risk. When cellular networks are unavailable — on remote hiking trails, at large outdoor festivals, or in disaster zones — standard messaging apps such as WhatsApp, Telegram, and Signal stop working entirely because they depend on the internet [3]. The ITU reports that 2.6 billion people worldwide still lack internet access [1], and even in well-connected regions, major disasters routinely destroy communication infrastructure within hours. During the 2010 Haiti earthquake, cellular networks failed almost immediately, directly increasing casualties and delaying rescue coordination [2].

People affected include hikers and trekkers in remote areas, event organizers coordinating large crowds, and emergency responders in disaster zones. Current offline tools do exist but each has critical gaps. GoTenna Mesh requires dedicated hardware not available on standard phones and has no intelligent routing logic [5]. Bridgefy uses Bluetooth mesh but was found to have serious security vulnerabilities including identity spoofing and man-in-the-middle attacks in a full audit [4]. Meshtastic uses LoRa radio technology, requiring extra hardware beyond a smartphone [8]. Critically, none of these tools use real-time device information — such as battery level, proximity, or network congestion — to make smarter routing decisions.

The proposed solution is a smartphone-based offline communication platform using Bluetooth Low Energy (BLE) mesh networking, multi-hop relay routing, and delay-tolerant store-and-forward messaging. The Context-Aware Communication Manager (CACM) is the central intelligence that coordinates all three communication methods. It selects the best path for each message based on live device conditions, with no internet or dedicated hardware required. This research aligns with SDG 3 (Good Health — emergency safety), SDG 9 (Infrastructure — resilient low-cost communication), and SDG 11 (Sustainable Cities — safe community coordination). It falls within the SST Research Cluster, specifically mobile networking and context-aware mobile systems.

1.2 Literature Review

1.2.1 Commercial Offline Messaging Tools

goTenna EdgeRelay sends text messages over a custom LoRa mesh network without internet connectivity, operating at long ranges but requiring expensive dedicated hardware with no context-aware routing, no emergency priority, and no battery-aware path selection [5]. Bridgefy gained widespread attention as a Bluetooth offline messenger, however a security audit revealed critical flaws including identity spoofing, man-in-the-middle vulnerability, and poor routing performance in low-density networks, requiring the platform to be completely redesigned [4]. Meshtastic is an open-source long-range messaging tool but requires LoRa radio modules that are not built into standard smartphones [8].

1.2.2 Delay Tolerant Networks and MANET Research

The DTN architecture has been formally described as a store-carry-forward model designed for networks where devices are not always connected, forming the theoretical foundation for all offline message delivery research [6]. Epidemic routing was introduced as an approach that copies messages to every nearby device to maximise delivery probability, but this method wastes battery and network bandwidth at scale [7]. Subsequent research improved upon this with probabilistic routing that uses contact history to forward messages more selectively [9]. While effective in research settings, these protocols were designed for infrastructure-level systems and do not account for battery constraints, real-time proximity, or message priority on mobile devices.

1.2.3 Bluetooth Low Energy Mesh Systems

A multipath-optimised AODV approach for BLE mesh networks has been shown to significantly enhance reliability and stability in scenarios without fixed infrastructure, confirming BLE mesh as a viable technology for emergency offline communication [10]. Further studies on BLE-based systems reviewed applications in localisation and real-world deployments, confirming that BLE supports reliable communication with low power consumption [11]. However, neither approach includes a coordination layer to intelligently select between different communication methods based on live device context.

1.2.4 Context-Aware and Intelligent Routing

Research into neuro-symbolic approaches for context-aware activity recognition on mobile devices has demonstrated that using real device state — including battery level, signal strength, and movement — to influence routing decisions significantly improves network performance and reliability [12]. A comparative analysis of machine learning integration in delay-tolerant networking showed that rule-based decision engines achieve comparable routing accuracy with a fraction of the processing overhead of neural network approaches [13]. This finding strongly supports the CACM design choice: a rule-based engine is both technically sufficient and battery-efficient for this application..

1.3 Research Gap

The literature confirms that no existing system — commercial or research — combines BLE mesh, multi-hop routing, and delay-tolerant transport with an intelligent coordination layer. The table below identifies five specific gaps and how this research addresses each:

CACM :- Context-Aware Communication Manager

Research Gap	How CACM Addresses It
No offline system has a coordination layer that dynamically selects between multiple communication methods	CACM uses a 5-rule decision engine to select BLE Direct, Multi-Hop, or DTN per message in real time
Existing tools ignore battery levels of relay devices when routing	CACM reads battery from Hello Packets; excludes any relay below 20%; prefers highest-battery path
No consumer offline app gives emergency messages higher priority than normal ones	CACM detects EMERGENCY flag, bypasses the queue, and broadcasts via all 3 layers at once
All current offline systems use fixed routing rules that cannot change with device conditions	CACM re-evaluates 5 context rules per message: proximity, battery, congestion, connectivity, priority
Heavy AI/ML models are not practical on battery-limited smartphones	CACM uses simple if-else rules: decision time < 200ms, extra battery use < 5% per hour

This table confirms the novelty of the CACM. The contribution is both technically new — no prior system includes this coordination layer — and practically creative: it applies context-aware computing to offline mobile mesh networking, a domain where it has not been systematically used before.

OBJECTIVE

2.1 Main Objective

To design, build, and test a lightweight Context-Aware Communication Manager (CACM) that intelligently coordinates offline message routing across BLE direct, multi-hop, and delay-tolerant communication layers on standard Android smartphones — without internet or AI models — and to prove it delivers measurably better routing efficiency, emergency response speed, and battery usage than static routing in outdoor and emergency settings.

2.2 Specific Objectives

1. **SO1:** To build a Proximity Monitoring Module that continuously reads BLE RSSI values, converts them to estimated distance (metres), and updates the routing table within 5 seconds of any device movement.
2. **SO2:** To implement a Decision Engine that evaluates 5 context rules per outgoing message — priority, proximity, battery, congestion, connectivity — and selects the correct routing channel within 200 milliseconds.
3. **SO3:** To build an Emergency Handler that detects EMERGENCY-flagged messages, bypasses the normal queue, triggers simultaneous broadcast via all 3 layers with TTL=50, and auto-re-broadcasts up to 3 times if no ACK is received within 30 seconds.
4. **SO4:** To implement Battery-Aware Relay Selection that reads battery levels from Hello Packets, excludes relays below 20%, and prefers the path with the highest average relay battery.
5. **SO5:** To design a Component Coordinator with API interfaces to all 3 communication layer components, and update the user interface with delivery status (SENT / IN_TRANSIT / DELIVERED / FAILED) within 1 second.
6. **SO6:** To evaluate the CACM on 4 physical Android devices across 3 test scenarios, measuring routing accuracy (≥ 90%), emergency latency (< 2 sec), and battery improvement (≥ 20%) compared to a static routing baseline.

METHODOLOGY

This research follows the System Development and Evaluation methodology from the IT4010 Research Methodologies Guide. This approach is chosen because a new software component is being built to solve a real engineering problem, performance is measurable through controlled tests, and a direct comparison with a static routing baseline is possible. The research claim to validate is: the CACM reduces emergency message delivery latency by at least 40% and improves routing efficiency by at least 20% compared to static routing on real Android devices.

Research Approach and Technical Methods

The CACM uses a rule-based decision engine — five prioritized if-then-else rules evaluated per message — rather than machine learning. Pham et al. (2025) demonstrated that rule-based engines match ML routing accuracy at under 5% of the processing cost on mobile hardware [13], making this the correct technical choice for a battery-constrained offline device.

BLE RSSI values are read continuously using the Android BLE GATT API (BluetoothLeScanner). Distance is estimated using the log-distance path loss model: d = 10^((TxPower − RSSI) / (10 × n)), with n = 2 for open outdoor spaces. Five consecutive readings are averaged per estimate to reduce noise. Device context is maintained in a SQLite Context Table storing UUID, estimated distance, battery level, hop count, and last-seen timestamp for every known device. Hello Packets are broadcast every 10 seconds by each device to share battery and presence information across the network.

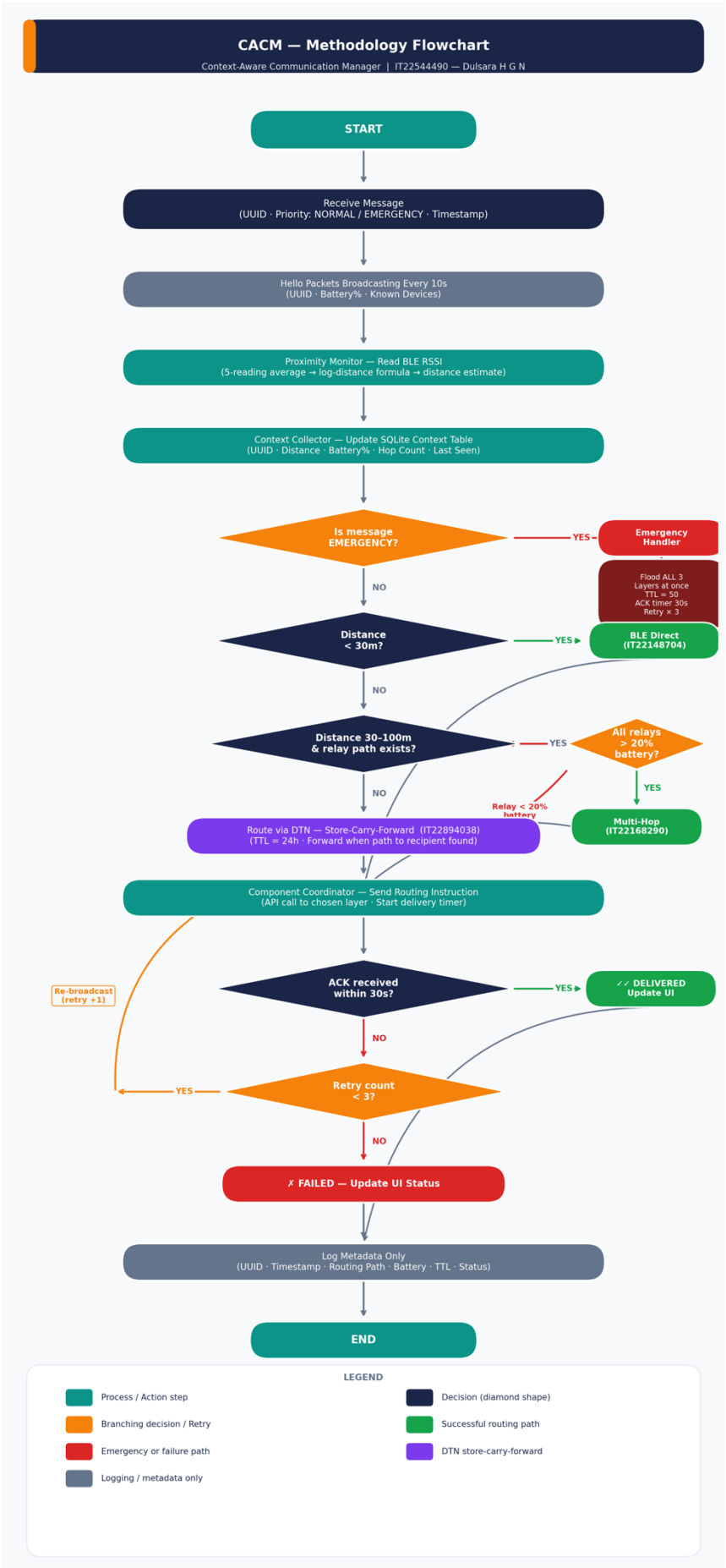

CACM — Methodology Flowchart
Context-Aware Communication Manager | IT22544490 — Dulsara H G N
START
Receive Message
(UUID · Priority: NORMAL / EMERGENCY · Timestamp)
Hello Packets Broadcasting Every 10s
(UUID · Battery% · Known Devices)
Proximity Monitor — Read BLE RSSI
(5-reading average → log-distance formula → distance estimate)
Context Collector — Update SQLite Context Table
(UUID · Distance · Battery% · Hop Count · Last Seen)
Is message EMERGENCY?
YES
Emergency Handler
Flood ALL 3 Layers at once
TTL = 50
ACK timer 30s
Retry × 3
NO
Distance < 30m?
YES
BLE Direct (IT22148704)
NO
Distance 30-100m & relay path exists?
YES
All relays > 20% battery?
YES
NO
Relay < 20% battery
Multi-Hop (IT22168290)
Route via DTN — Store-Carry-Forward (IT22894038)
(TTL = 24h · Forward when path to recipient found)
Component Coordinator — Send Routing Instruction
(API call to chosen layer · Start delivery timer)
ACK received within 30s?
YES
✓✓ DELIVERED Update UI
Re-broadcast (retry +1)
NO
YES
Retry count < 3?
NO
✗ FAILED — Update UI Status
Log Metadata Only
(UUID · Timestamp · Routing Path · Battery · TTL · Status)
END
LEGEND
Process / Action step
Decision (diamond shape)
Branching decision / Retry
Successful routing path
Emergency or failure path
DTN store-carry-forward
Logging / metadata only

Figure 3 (Methodology): CACM Decision Engine Flowchart — Context-Aware Communication Manager

Tools and Technologies

Technology	Role	Justification	Alternative Considered
Android (API 26+, Kotlin)	Application platform	Largest global smartphone market share (72%, StatCounter 2024); native BLE GATT APIs; Kotlin coroutines for async BLE operations; no cross-platform overhead.	Flutter / React Native - excluded: indirect BLE access via platform channels, higher battery overhead, unreliable background scanning
Android BLE GATT API	BLE communication	Native framework; no third-party dependency; supports Central+Peripheral simultaneous mode (API 21+).	Nordic BLE Library - adds dependency without meaningful advantage for this use case
Room Database (SQLite)	Routing table & message cache persistence	Lightweight, Android-native; supports LiveData for reactive UI updates; zero external server dependency.	Firebase (requires internet); Realm (heavyweight ORM)
AES-256-GCM (javax.crypto)	End-to-end message encryption	NIST-standard; hardware-accelerated on Android 6+; authenticated encryption prevents tampering without separate HMAC.	ChaCha20-Poly1305 - considered but less hardware support on older devices
Android Studio + Espresso	IDE and UI/integration testing	Official Android toolchain; Espresso enables instrumented BLE integration tests on physical devices.	IntelliJ - same engine but Android Studio has better emulator/device tooling
nRF Sniffer + Wireshark	BLE packet validation	Captures raw BLE advertising and GATT traffic for hop verification and security validation.	Bluetooth HCI logging - limited to single device perspective
Python (NetworkX)	Routing simulation	Rapid pre-implementation algorithm validation; NetworkX provides graph topology simulation for RREQ/RREP flows.	MATLAB - licensed cost; less community routing simulation support

Data Collection and Ethical Considerations

Testing uses physical Android smartphones in three controlled outdoor scenarios: small group (3–5 devices, < 30m), medium group (6–8 devices, 10–120m mixed), and emergency simulation (6 devices, 2 forced to < 20% battery, one emergency message). Each scenario runs five times for statistical validity. Logs record only metadata: UUID, timestamp, routing path, TTL, delivery status, and battery at routing time. No message content is ever stored. No

sensitive personal data is collected. Participants provide informed consent. SLIIT institutional research guidelines are followed throughout.

Validation and Performance Metrics

Metric	How Measured	Target
Routing Decision Accuracy	% messages routed via correct layer (log vs ground truth)	≥ 90%
Emergency Delivery Latency	Time (ms) from send to first hop receipt	< 2,000 ms
Battery Efficiency Improvement	mAh per 100 messages WITH vs WITHOUT CACM	≥ 20% reduction
Message Delivery Rate	% messages delivered across all test scenarios	≥ 85%
Decision Engine Speed	Milliseconds per routing decision	< 200 ms

Research Approach

This research adopts the Design Science Research (DSR) paradigm as its overarching methodology. DSR is appropriate for engineering-oriented IT research where the primary output is a purposefully constructed software artefact. The six DSR activities map directly onto the project phases: (1) Problem Identification — the gap is established in the literature review; (2) Solution Objectives — measurable targets are derived in Section 2.2; (3) Design and Development — the five-rule decision engine, proximity monitor, and component coordinator are designed and implemented in Kotlin; (4) Demonstration — the prototype is deployed on 4 physical Android smartphones in three controlled test environments; (5) Evaluation — quantitative metrics are measured against pre-defined acceptance criteria; (6) Communication — findings are disseminated through the research dissertation. DSR is preferred over action research or case study because the project objective is to construct and empirically test a tangible software component.

Research Assumptions

The following assumptions underpin the design decisions and experimental scope of this study:

Assumption	Description
Device homogeneity	All test nodes are Android smartphones (API 26+) capable of simultaneous BLE Central and Peripheral operation.
Pre-installed application	All communicating devices run the same application version; version heterogeneity is out of scope for Phase 1.
BLE range adequacy	Adjacent hops are within nominal BLE range (10–50 m indoors, up to 100 m line-of-sight outdoors) such that GATT connections can be established.
Cooperative relay nodes	Intermediate relay nodes are assumed to be cooperative participants. Adversarial relay nodes that selectively drop packets are out of scope.

Static topology during RSSI sampling	Devices remain within BLE range of their immediate neighbours for the 5-second RSSI averaging window used by the Proximity Monitor.
Android BLE API consistency	The Android BLE GATT API behaves consistently across the target API range (26–34); manufacturer-specific quirks are handled through a device-compatibility test matrix.

Technical Constraints

The following technical constraints define the operating envelope of the implemented system:

Constraint	Detail
Max simultaneous GATT connections	Android BLE stacks support a practical maximum of 7 simultaneous GATT connections per device, bounding the fan-out of the Proximity Monitor.
BLE advertising payload size	BLE advertising packets are limited to 31 bytes (legacy) or 254 bytes (Extended Advertising, API 26+). Hello Packets must fit within this payload.
Android background BLE restrictions	Android 8.0+ throttles background BLE scan intervals to 30 s. The application must hold a foreground service notification to maintain continuous scanning.
Routing table capacity	The Context Table handles up to 100 simultaneous known devices; entries unseen for >60 s are auto-purged to bound memory usage on 2 GB RAM devices.
Android-only scope	The implementation is Android-only (Kotlin, API 26+). iOS devices cannot participate as routing nodes in Phase 1.
AES-256-GCM overhead	Encryption adds a 12-byte IV and 16-byte authentication tag per message plus ~1–3 ms processing latency on mid-range Android devices.

Limitations

The following limitations define the conditions under which the conclusions of this research are valid:

Limitation	Impact
Small-scale test network (4 devices)	Behaviour at larger scales (20–100 nodes) is estimated through Python/NetworkX simulation but not empirically verified on physical hardware.
Controlled environment testing	Real outdoor disaster scenarios involve unpredictable RF interference and crowd density that may degrade performance below target thresholds.
Single-application ecosystem	All nodes must run the same application version. Backward-compatible versioning across versions is deferred to future work.
Battery measurement granularity	Android BatteryManager API reports in 1% increments. Fine-grained mWh-level measurement requires hardware power monitors not available in this study.
Single-component isolation	CACM performance metrics are measured in isolation. Integration-level performance under full four-component operation may differ and is evaluated in the group integration phase.
No quality-of-service beyond emergency flag	All non-emergency messages are treated with equal priority. Fine-grained QoS tiering is noted as a future enhancement.

HIGH-LEVEL SYSTEM ARCHITECTURE

The platform integrates four components built by four team members. The CACM (IT22544490) sits at the centre and coordinates all three communication layers. The table below summarises each component:

Student ID	Component	Responsibility
IT22148704	Short-Range Direct Communication	BLE 1-to-1 and group messaging, delivery confirmations, encryption (0–25m)
IT22168290	Decentralised Multi-Hop Routing	Message forwarding through relay nodes, end-to-end acknowledgements (25–75m)
IT22544490 ★	Context-Aware Communication Manager	Central intelligence: 5-rule routing engine, emergency handler, battery relay selection, all-component API coordination
IT22894038	Delay-Tolerant Opportunistic Transport	Local message storage, opportunistic forwarding, TTL management (75m+)

Figure 1 shows the overall system architecture with the CACM at the centre. Figure 2 shows the CACM internal architecture.

	USER (Mobile App) — Composes message, selects recipient, sets priority (Normal / Emergency)			
		↓ Message Input		
★ CONTEXT-AWARE COMMUNICATION MANAGER (IT22544490) ★ **[Proximity Monitor] → [Context Collector] → [Decision Engine] → [Emergency Handler] → [Coordinator]** **Reads proximity, battery, connectivity → Selects routing path → Coordinates all 3 layers**				
	↓ BLE Direct	↓ Multi-Hop	↓ DTN Store	
	BLE DIRECT IT22148704 0–25m	**MULTI-HOP** IT22168290 25–75m	**DTN STORE** IT22894038 75m+	
	✓ Instant delivery	✓ Hop-by-hop delivery	✓ Eventual delivery	
All messages are end-to-end encrypted | CACM reads metadata only | No internet required				

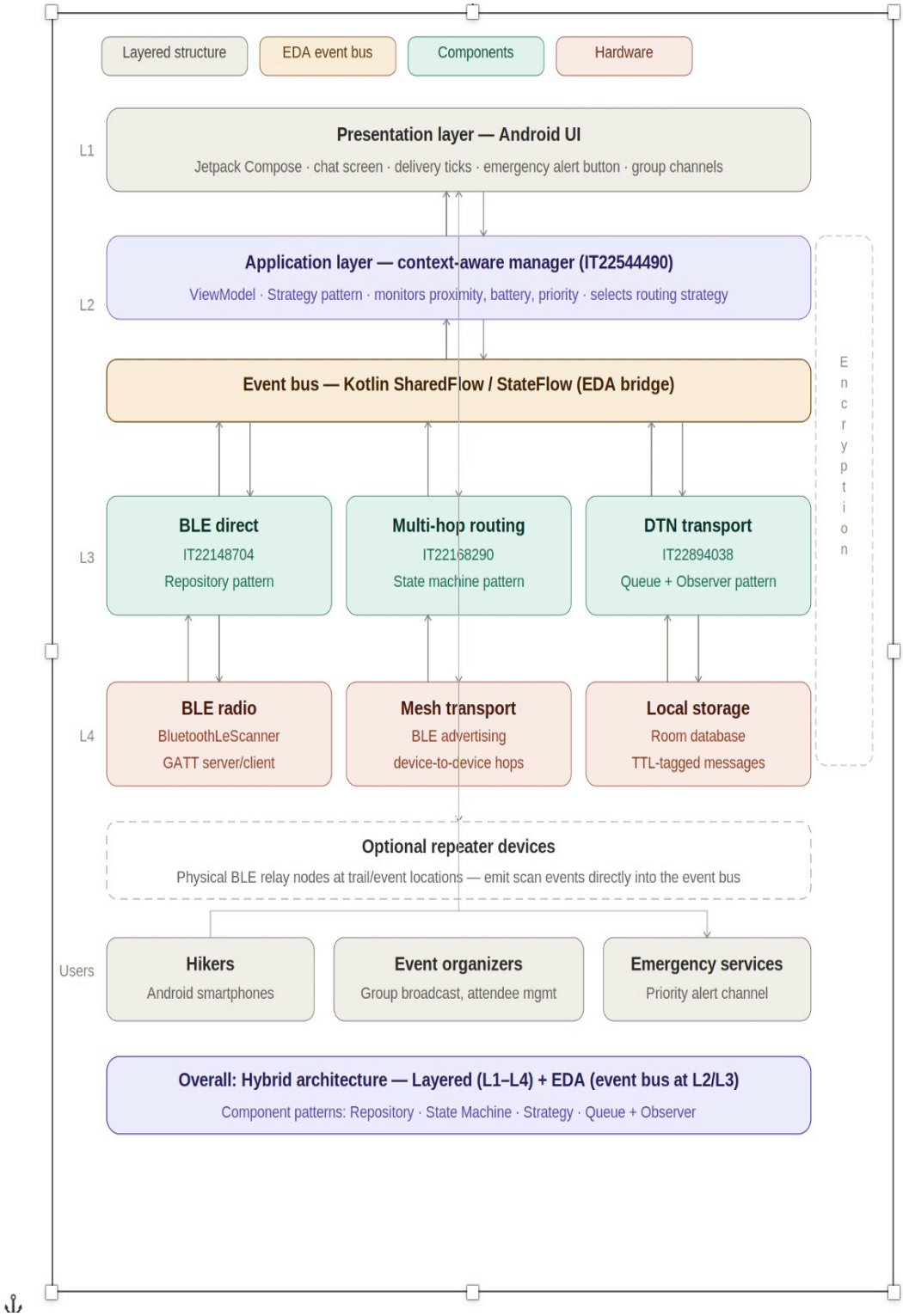


Figure 1: Overall System Architecture — Offline-First Communication Platform

▼ INPUTS ▼		
BLE RSSI Scanner (Proximity Monitor) Measures signal strength → estimates distance	Context Collector Reads battery level Parses Hello Packets → builds Context Table	Connectivity Monitor Checks internet / LTE Updates status flags → notifies UI
↓ All context fed into Decision Engine ↓		
DECISION ENGINE (Rule-Based — 5 Priority Rules) **Rule 1: EMERGENCY? → all 3 layers simultaneously, TTL=50, 30s ACK timer** **Rule 2: Distance < 30m → BLE Direct | 30–100m → Multi-Hop | >100m → DTN** **Rule 3: Relay battery < 20% → skip that relay, use alternate path** **Rule 4: Queue > 50 msgs → pause normal, process emergency first** **Rule 5: Internet found → notify user (no auto-switch to online)**		
↓ Decision sent to ↓		
EMERGENCY HANDLER **Bypasses queue → floods all 3 layers → TTL=50 → 30s ACK timer → auto re-broadcast if no ACK**		
COMPONENT COORDINATOR **Sends instructions via API → receives SENT / IN_TRANSIT / DELIVERED / FAILED → updates UI within 1 second**		
▼ OUTPUTS ▼		
BLE Direct (IT22148704) Instant delivery 0–**25**m	Multi-Hop (IT22168290) Hop delivery 25–1m	DTN (IT22894038) Store & forward 75m+
↕ Delivery status feedback → Coordinator → UI (✓ Sent / ⏳ In Transit / ✓✓ Delivered / ✗ Failed)		

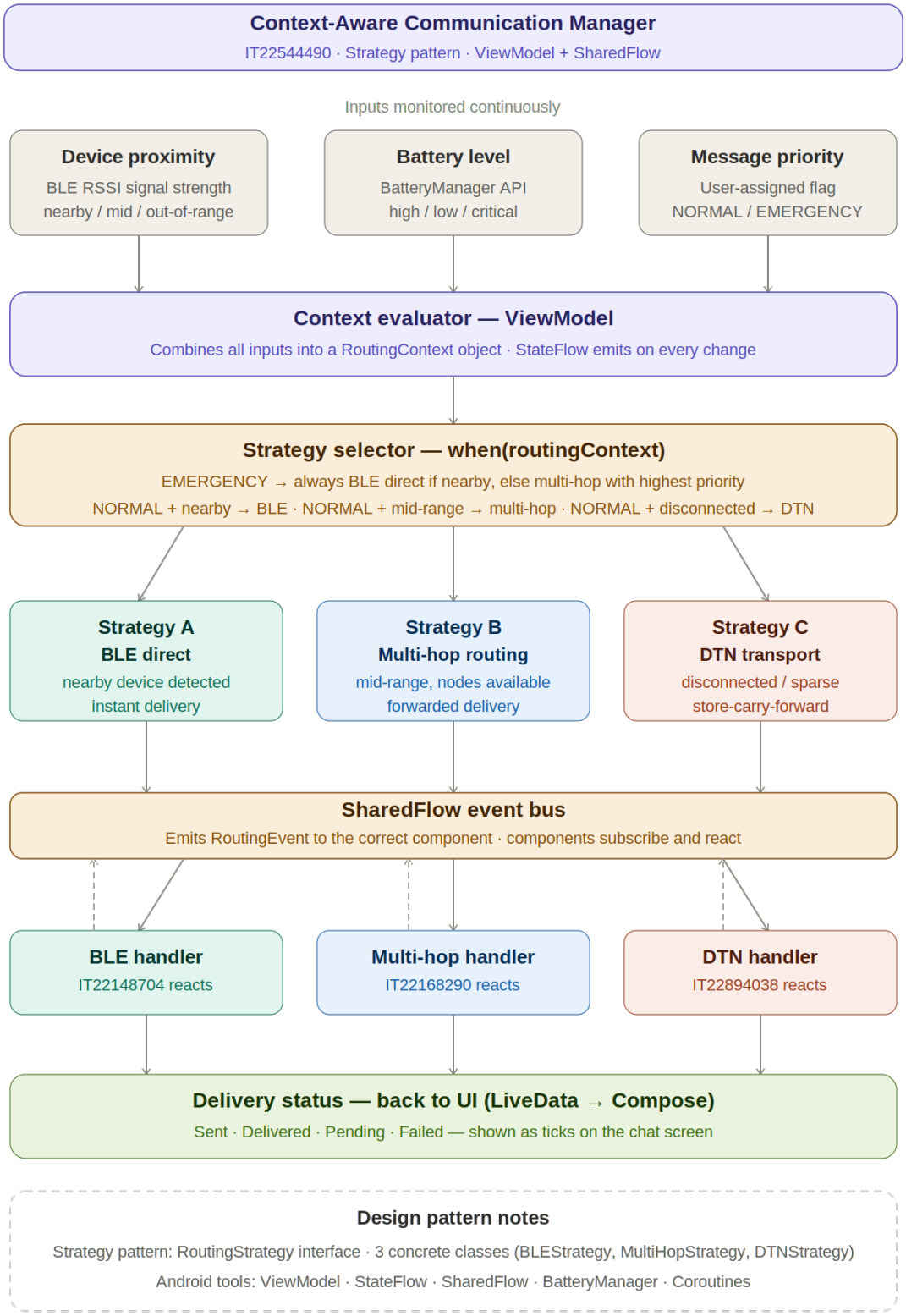


Figure 2: Context-Aware Communication Manager — Internal Architecture

Component Interactions and Data Flow

Every outgoing message follows this path: (1) User sends message with priority and recipient UUID. (2) Proximity Monitor provides distance estimate to recipient. (3) Context Collector retrieves relay battery levels from the Context Table. (4) Decision Engine applies 5 rules and selects BLE_DIRECT, MULTIHOP, or DTN. (5) Component Coordinator sends the routing instruction to the chosen layer. (6) Delivery status callback (SENT / IN_TRANSIT / DELIVERED / FAILED) is received and shown in the UI within 1 second.

Scalability and Security

The routing table handles up to 100 devices with O(1) UUID lookup via SQLite indexing. Stale entries (not seen in > 60 seconds) are auto-purged. The CACM never reads or stores message content — routing decisions use metadata only. Message content is end-to-end encrypted before it reaches the CACM. If any component is unreachable, the CACM falls back to the next available routing option automatically with no single point of failure.

USER REQUIREMENTS

Stakeholder Identification

- Primary Users: Hikers, trekkers, and outdoor adventure groups in areas without cellular coverage.
- Secondary Users: Event organizers and coordination staff at outdoor festivals and corporate events.
- Emergency Users: Rescue workers and disaster response teams in infrastructure-damaged zones.
- System Developers: The project team integrating the CACM with the three communication components.

Functional Requirements

FR#	Functional Requirement
FR1	The CACM shall continuously scan for nearby BLE devices and estimate proximity using RSSI, updating the routing table within 5 seconds of any change.
FR2	The CACM shall read the device battery level and parse Hello Packets from neighbours to extract their battery and hop-count data.
FR3	The CACM shall route messages to recipients within 30m via the BLE Direct component (IT22148704).
FR4	The CACM shall route messages to recipients 30–100m away via the Multi-Hop Routing component (IT22168290).
FR5	The CACM shall route messages beyond 100m or to unreachable recipients via the DTN component (IT22894038).
FR6	The CACM shall detect EMERGENCY messages and transmit them simultaneously via all 3 layers, bypassing the normal queue.

FR7	The CACM shall exclude relay nodes with battery < 20% from multi-hop paths and prefer the path with the highest average relay battery.
FR8	The CACM shall provide API interfaces to all 3 communication components with documented request and response formats.
FR9	The CACM shall collect delivery status from all components and update the UI within 1 second of any status change.
FR10	The CACM shall pause non-emergency messages when queue > 50 and resume in timestamp order once all emergency messages are delivered.
FR11	The CACM shall start a 30-second ACK timer per emergency message and auto-re-broadcast (max 3 times) if no ACK is received.
FR12	The CACM shall notify the user when internet is detected without automatically switching from offline mode.

Non-Functional Requirements

Category	Requirement
Performance	Routing decisions computed within 200ms; UI status updates within 1 second of any callback.
Battery	CACM background processes (BLE scanning, Context Table updates) use < 5% extra battery per hour on mid-range Android.
Reliability	Decision Engine produces a valid routing decision for 100% of messages with no silent failures or lost messages.
Scalability	Context Table and routing table handle up to 100 simultaneous known devices without slowing down on 2GB RAM devices.
Usability	Status indicators (✓ Sent / ⏳ In Transit / ✓✓ Delivered / ✗ Failed) shown clearly with no technical jargon visible to users.
Security	CACM never reads, logs, or stores message content. All routing decisions use metadata only. No routing data leaves the local mesh.
Maintainability	Each sub-module independently testable with mock inputs without needing the full system running.
Portability	Runs on Android API level 21 (Android 5.0) and above with no code changes.

COMMERCIALIZATION PLAN

The target market includes outdoor recreation businesses (hiking tour operators, adventure sports companies), event management companies organizing outdoor festivals, and emergency management organizations. The core value proposition is the only offline smartphone communication tool that routes every message intelligently based on real-time device conditions — with no extra hardware, no internet, and no AI server required. The emergency broadcast feature is a key differentiator: no other consumer offline tool automatically prioritizes life-safety messages above all traffic and retries delivery until acknowledged.

Revenue Model and Pricing Strategy

- Freemium App: Free tier for basic BLE direct messaging. Premium at LKR 500/month per user activates full CACM features — context-aware routing, emergency broadcast, battery-aware relay selection.
- Enterprise License: LKR 60,000/year for up to 50 devices — targeting event companies and rescue teams. Includes onboarding and support.
- Hardware Bundle: Optional BLE repeater device (Raspberry Pi + BLE module) at LKR 18,000/unit to extend mesh range at fixed points such as event checkpoints.
- SDK License: CACM rule engine available to third-party developers as a paid SDK with royalties per integration.

Competitive Advantage

Feature	This System	GoTenna	Bridgefy	Meshtastic
No dedicated hardware needed	✅	❌	✅	❌
Context-aware intelligent routing	✅	❌	❌	❌
Emergency message prioritization	✅	❌	❌	❌
Battery-aware relay selection	✅	❌	❌	❌
Proven security (E2E encrypted)	✅	⚠️ Limited	❌ Vulnerabilities	✅
Works on any Android phone	✅	❌	✅	❌

Cost estimation shows that all development tools are open-source (zero licensing cost), keeping build costs minimal. The main ongoing costs are cloud backend hosting for the freemium app and customer support. The CACM rule engine design, emergency broadcast protocol, and battery-aware relay algorithm represent novel technical contributions that can be protected as trade secrets or filed for software patent consideration under SLIIT's undergraduate IP policy.

BUDGET AND JUSTIFICATION

Category	Item	Cost (LKR)	Justification
Software	Android Studio, SQLite, Python, Git (all open-source)	0	No licensing fees required
Hardware	2 extra Android phones for BLE testing	12,000	BLE cannot be emulated; physical devices required
Hardware	USB-C cables and OTG adapters	1,500	ADB debugging and log capture during tests
Cloud	Google Colab Pro × 1 month (log analysis)	2,000	Python data analysis and graph generation
Other	Mobile data for development (3 months)	3,000	Library downloads, GitHub sync, team comms
Other	Printing (reports and supervisor copies)	2,500	Hard copies for submission and review
Contingency	Reserve for unexpected hardware issues	3,000	10% contingency for hardware-dependent project
TOTAL		**24,000**	

The budget is kept minimal because all development tools are free and open-source. The main cost is physical hardware for BLE testing (LKR 12,000), which cannot be replaced by software simulation. All cloud processing fits within free tiers except for one month of Colab Pro for data analysis.

WORK BREAKDOWN STRUCTURE (WBS)

The table below shows all tasks broken down by phase across the 18-week semester. Each task has a named deliverable and the workload is distributed to match the available time per phase.

Phase	Task / Activity	Deliverable
Phase 1 Planning Wk 1–2	▸ Literature review — 12 sources on BLE mesh, DTN, context-aware routing	Annotated bibliography, gap analysis
	▸ Internal API design with all 3 team members; FR/NFR list	API spec, requirements document
	▸ Decision tree design — 5 rules with all edge cases	Rule pseudocode, decision tree
Phase 2 Design Wk 3–4	▸ Proximity Monitor — RSSI-to-distance mapping algorithm design	Module design spec
	▸ Context Collector — Hello Packet format and Context Table schema	Hello Packet format doc
Phase 3 Dev Wk 5–12	▸ Implement Proximity Monitor (BLE RSSI scanning + distance estimation)	Working module, unit tests
	▸ Implement Context Collector (battery reader + Hello Packet parser)	Live Context Table, tests
	▸ Implement Decision Engine (5-rule routing selector)	Rules verified vs test cases
	▸ Implement Emergency Handler (flood broadcast + ACK timer)	Emergency delivery < 2 sec
	▸ Implement Component Coordinator (API links to 3 components)	All 3 layers routing correctly
Phase 4 Testing Wk 13–16	▸ Unit testing all sub-modules with mock inputs (> 95% coverage)	All unit tests pass
	▸ Integration testing with full team — all routing scenarios	Integration test report
	▸ Performance evaluation — routing accuracy, latency, battery	Metrics report with graphs
Phase 5 Docs Wk 17–18	▸ Write proposal report, methodology documentation	Submitted proposal
	▸ Final report and presentation preparation	Final report + slides

Figure 3: Work Breakdown Structure (WBS) — Context-Aware Communication Manager (IT22544490)

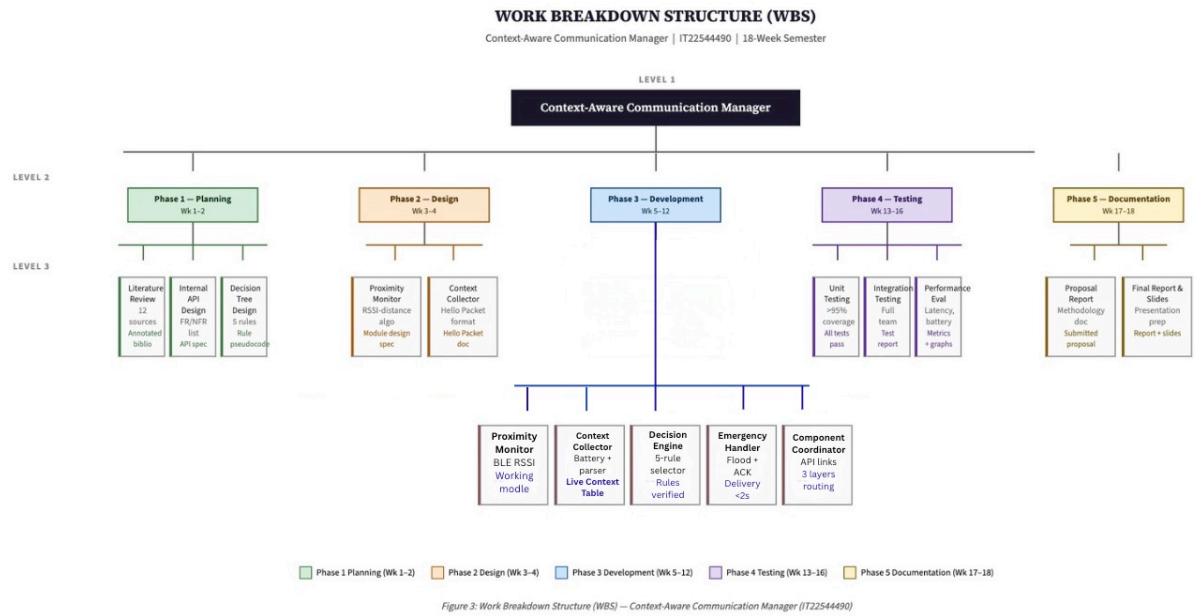


Figure 3: Work Breakdown Structure (WBS) — Context-Aware Communication Manager (IT22544490)

GANTT CHART

The chart below shows the full 18-week project timeline from March 2026 to July 2026, with coloured bars showing active development weeks and milestone alignment with semester checkpoints.

Task	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Month →	Mar	Mar	Mar	Mar	Apr	Apr	Apr	Apr	May	May	May	May	Jun	Jun	Jun	Jun	Jul	Jul
1. Literature Review	■	■																
2. Requirements & Design			■	■														
3. Proximity Monitor					■	■												
4. Context Collector							■											
5. Decision Engine								■	■									
6. Emergency Handler										■								
7. Component Coordinator											■	■						
8. Integration Testing													■	■	■			
9. Evaluation & Metrics															■	■		
10. Report & Documentation																	■	■

March 2026 – July 2026 | 18 Weeks | ■ = Active week

Figure 4: Project Gantt Chart — March 2026 to July 2026

REFERENCES LIST

[1] International Telecommunication Union, "Measuring Digital Development: Facts and Figures 2024," ITU, Geneva, Switzerland, 2024.

[2] A. Usmonov et al., "Resilient Mobility Protocols for Disaster Recovery Communication Networks," *Journal of Intelligent Systems and Internet of Things*, vol. 14, no. 4, pp. 1–18, 2025.

[3] WhatsApp Inc., "WhatsApp Security White Paper," Meta Platforms, Menlo Park, CA, USA, 2024.

[4] S. Albrecht et al., "Security Vulnerabilities in Offline-First Messaging Applications for Disaster Scenarios," in *Proc. 33rd USENIX Security Symposium*, Aug. 2024, pp. 1–16.

[5] goTenna, Inc., "goTenna EdgeRelay: Extended Off-Grid Mesh Network Coverage," goTenna Technical Brief, Jersey City, NJ, USA, 2024.

[6] G. Koukis, K. Safouri, and V. Tsaoussidis, "All About Delay-Tolerant Networking (DTN) Contributions to Future Internet," *Future Internet*, vol. 16, no. 4, art. 129, 2024.

[7] M. Jesus-Azabal et al., "Adaptive Epidemic Routing for Opportunistic Networks in Disaster-Affected Urban Environments," *Peer-to-Peer Networking and Applications*, vol. 17, no. 2, pp. 410–425, 2024.

[8] S. Cass, "It's Meshtastic! LoRa-Based Tech Brings Mesh Radio to Makers," *IEEE Spectrum*, vol. 61, pp. 16–18, 2024.

[9] F. M. S. Saif, P. H. Katroju, and R. Kassouf-Short, "A Constructive Analysis on Machine Learning Integration in High Delay Tolerant Networking (HDTN)," in *Proc. 7th IEEE International Conference on Advanced Communication Technologies and Networking (CommNet)*, Dec. 2024, pp. 1–6.

[10] M. R. Ghori et al., "Enhancing Reliability and Stability of BLE Mesh Networks: A Multipath Optimized AODV Approach," *Sensors*, vol. 24, no. 18, art. 5893, 2024.

[11] F. Milano et al., "BLE-Based Indoor Localization: Analysis of Solutions for Performance Improvement," *Sensors*, vol. 24, no. 2, art. 376, 2024.

[12] S. Arrotta, G. Civitarese, and C. Bettini, "Semantic Loss: A Neuro-Symbolic Approach for Context-Aware Human Activity Recognition on Mobile Devices," *Proc. ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 7, no. 4, pp. 1–29, 2024.

[13] D. V. Pham et al., "Q-Learning-Based Forwarding Routing for Delay Tolerant Networks in Disaster Scenarios," *CommIT Journal*, vol. 19, no. 1, pp. 1–14, 2025

APPENDICES

Appendix A: Decision Engine Step-by-Step Flowchart

1. Receive message: encrypted content, recipient UUID, priority (NORMAL / EMERGENCY), timestamp.
2. Is priority EMERGENCY? YES → Emergency Handler: send via all 3 layers simultaneously, TTL=50, start 30-second ACK timer. Auto-re-broadcast maximum 3 times if no ACK received. END.
3. NO → Look up recipient UUID in Context Table. Get estimated distance.
4. Distance < 30m? YES → Route via BLE_DIRECT (IT22148704). Go to Step 7.
5. Distance 30–100m AND relay path exists? Check relay batteries. All relays > 20%? YES → Route via MULTIHOP (IT22168290). Go to Step 7.
6. Any relay < 20% battery? Find alternate path. No valid path found? → Fall through to DTN.
7. Route via DTN (IT22894038). Store with TTL = 24 hours. Forward when path to recipient found.
8. Send routing instruction to Component Coordinator. Await delivery status callback. Update UI: ✓ Sent / ⏳ In Transit / ✓✓ Delivered / ✗ Failed.

Appendix B: Hello Packet Structure

Hello Packets are broadcast by every active device every 10 seconds over BLE. Fields:

- senderUUID — 128-bit unique device ID (generated locally on first app launch; never changes)
- displayName — Human-readable name set by user during setup (for UI only; not used in routing)
- batteryLevel — Current battery percentage (integer 0–100; read from Android BatteryManager API)
- knownDevices — Array of { uuid, estimatedHops } for all devices this sender can currently reach
- timestamp — Unix timestamp in milliseconds; entries older than 60 seconds are auto-removed from Context Table

Appendix C: Risk Register

The table below identifies the key risks to this research project, their assessed likelihood and impact, and the planned mitigation and contingency strategies.

Risk Description	Likelihood	Impact	Score	Mitigation Strategy	Contingency Plan
Android BLE API inconsistencies across device manufacturers cause GATT communication failures.	High	High	Critical	Test on 3+ device brands from Week 1; implement a BLE abstraction layer isolating manufacturer-specific quirks.	Restrict deployment to tested device models; document compatibility matrix.
BLE signal range insufficient for outdoor multi-hop test scenarios.	Medium	High	High	Conduct preliminary range tests in Week 1; supplement with optional BLE repeater devices at fixed locations.	Reduce target network size; focus validation on 3–5 device scenarios.
Context Table overload or stale entries degrade routing decisions under high device density.	Medium	Medium	Medium	Implement auto-purge for entries unseen >60 s from Day 1; unit test with 100-device mock table.	Lower purge threshold to 30 s; cap table at 50 entries with LRU eviction.
Integration delays with peer components due to API contract changes.	Medium	Medium	Medium	Freeze inter-component API spec at Phase 1; maintain stub implementations for independent testing.	Develop mock API implementations for each peer component; defer integration to Phase 4.
Battery overhead of continuous BLE scanning exceeds 5% NFR target.	Medium	Low	Low	Implement adaptive scan intervals (reduce frequency when no active routing); profile battery from Phase 3.	Adjust NFR target based on empirical data; document as known limitation.
Emergency handler ACK timer race condition causes duplicate re-broadcasts.	Low	Medium	Low	Use Kotlin StateFlow to manage ACK state; unit test with simulated delayed ACKs and concurrent messages.	Set conservative retry limit (max 3); log all re-broadcasts for post-test analysis.

Appendix D: Inter-Component API Specification (v1.0)

The Kotlin interface below defines how the three peer components (IT22148704, IT22168290, IT22894038) interact with the Context-Aware Communication Manager. All methods are called on the main thread via coroutine dispatch; callbacks are delivered on the UI thread using LiveData.

CACM Outbound API (CACM calls peer components)

```
interface BLEDirectAPI {
    fun sendDirect(message: Message, recipientUUID: String,
                   onStatus: (DeliveryStatus) -> Unit)
    fun cancelSend(messageId: String)
}

interface MultiHopRoutingAPI {
    fun routeMessage(message: Message, destinationId: String,
                     onStatusUpdate: (DeliveryStatus) -> Unit)
    fun cancelRoute(messageId: String)
    fun getRoutingMetrics(): RoutingMetrics
}

interface DTNTransportAPI {
    fun storeAndForward(message: Message, recipientUUID: String,
                        ttlHours: Int, onStatus: (DeliveryStatus) -> Unit)
    fun cancelDelivery(messageId: String)
}
```

CACM Inbound API (peer components call CACM)

```
interface CACMInboundAPI {
    /** Called by any layer to report delivery status change */
    fun onDeliveryStatus(messageId: String, status: DeliveryStatus)

    /** Called by Proximity Monitor with updated RSSI reading */
    fun onRSSIUpdate(deviceUUID: String, rssiDbm: Int)

    /** Called by Hello Packet parser with neighbour context */
    fun onHelloPacket(senderUUID: String, batteryPct: Int,
                      hopCount: Int, timestamp: Long)
}

enum class DeliveryStatus { SENDING, SENT, IN_TRANSIT,
                            DELIVERED, FAILED }

data class RoutingMetrics(val activeRoutes: Int,
    val avgHopCount: Float, val cacheSize: Int,
    val pendingACKs: Int)
```

All API methods are documented with KDoc comments in the codebase and versioned using semantic versioning (v1.0.0). Breaking changes require a major version increment and team sign-off.